



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Robotics and Automation Engineering

Filtering and Identification of Dynamical Systems

Kalman smoother : Theory and Case of application

Professori:

Prof. Giuseppe Fedele

Ing. Luigi D'Alfonso

Studentessa:

Francesca Vaccaro

Mat. 276582

ANNO ACCADEMICO 25/26

Indice

Indice	1
1 Introduction	2
1.1 Why do we need a kalman smoother	2
1.2 Background and Starting point	3
1.2.1 Kalman filter algorithm	5
2 What is the kalman smoother	7
2.1 The Concept of Smoothing	7
2.2 The Three Types of Smoothing Problems	8
3 Algorithm and Implementation	9
3.1 The Rauch–Tung–Striebel (RTS) Smoother: A Bayesian Per- spective	9
3.1.1 The RTS Algorithm Formulation	10
4 Pratical Project	12
4.1 Overview	13
4.1.1 Data download and cleaning	13
4.1.2 Implementation of kalman filter	15
4.1.3 Implementation of the smoother	18
4.1.4 Graph discussion	19
5 Conclusion	22
Bibliografia	23

Capitolo 1

Introduction

In signal processing, a fundamental goal is to infer the hidden state of a dynamic system from observations subject to both process and measurement noise.

Among the various methodologies developed to address this challenge, the Kalman filter stands out as a standard solution, providing optimal state estimates recursively as new data is processed.

1.1 Why do we need a kalman smoother

Although the Kalman filter is one of the most effective estimation techniques for linear dynamic systems affected by Gaussian noise, it has an intrinsic limitation related to its causal nature. As already observed in the study of estimation and prediction problems, a causal algorithm can base its decisions exclusively on information available up to the current time. In other words, the estimate of the state at time k is obtained using only observations acquired up to that time, without being able to exploit data that will be available in the future.

This characteristic makes the Kalman filter particularly suitable for real-time applications, where measurements are processed as they are acquired. However, there are numerous contexts in which the entire sequence of observations is already available before processing, such as in the offline analysis of experimental data or recorded biomedical signals.

In such situations, the quality of the estimate can be further improved by also exploiting future observations. This is precisely the goal of the Kalman Smoother, which does not replace the Kalman filter but is an extension of it. The smoother initially performs a standard forward filtering process, obtaining causal estimates of the state, and then performs a second processing backward in time, during which each estimate is refined using information from subsequent time points.

Using future observations reduces the uncertainty associated with the estimates and achieves a more accurate reconstruction of the system dynamics than that provided by the Kalman filter alone. For this reason, smoothing algorithms are used in numerous offline applications, where a real-time response is not required but maximizing the accuracy of the estimate is essential.

In this project, this approach will be applied to the analysis of electroencephalographic (EEG) signals. In this case, the Kalman filter and the Rauch–Tung–Striebel smoother are not used to directly filter the signal, but rather to more accurately estimate the coefficients of a time-varying autoregressive model describing its evolution. The quality of these coefficients directly influences the reconstruction of the signal spectrum and, consequently, the possibility of correctly analyzing transient phenomena such as Event-Related Desynchronization (ERD) and Event-Related Synchronization (ERS).

1.2 Background and Starting point

To ensure a more linear progression, the starting point of this discussion is precisely the Kalman filter. Conceptually, it behaves as a sort of Luenberger observer, meaning its purpose is to estimate the state from past and present measurements. However, it does not stop there: unlike the Luenberger observer, the Kalman filter also operates on systems affected by both process and measurement noise. Furthermore, the term ‘filter’ stems from the fact that it is not limited to state estimation, but it also estimates the output.

Its computational power is also evident in applications where signals are not stationary processes (WSS), and there are specific adaptations that enable it to operate on non-linear systems (such as the EKF).

Let's start from the definition of a system of this type:

$$\begin{cases} x(t+1) = Fx(t) + v_1(t) \\ y(t) = Hx(t) + v_2(t) \end{cases} \quad (1.1)$$

Where $v_1(t)$ e $v_2(t)$ are the errors referred respectively to the process and the measurements, both modeled as gaussian noise with 0 mean and V_1 and V_2 matrices of variance.

Other assumptions :

1. $E[x(1)] = 0$ the expected value of the first element.
2. $Var[x(1)] = P(1)$ the variance.

If we take the first consideration and the fact that also the expected value of the noise is equal to 0(because is a gaussian noise), we arrive at the consideration that $E[x(t)] = 0$ for each time instant t , and also $E[y(t)] = 0$.

Overall the system is described by the $n \times n$ dynamic matrix F , the $p \times n$ output matrix H , the $n \times n$ variance matrix V_1 of $v_1()$, the $p \times p$ variance matrix V_2 of $v_2()$, and the variance matrix $P(1)$ of the initial state. All such matrices are assumed to be known. In its general form, the problem we address is as follows:

Kalman problem : Estimate $x(N+r)$, given the observations of $y(t)$ for all time points from $t = 1$ to time $t = N$. Our path will be articulated in various steps as follows: **One-step-ahead prediction problem** : $r = 1$ Estimate $x(N+1)$, given the observations of $y(t)$ from $t = 1$ to time $t = N$. **Multi-steps-ahead prediction problem** : $r \geq 1$ Estimate $x(N+r)$, given the observations of $y(t)$ from $t = 1$ to time $t = N$. Positive integer r is called prediction horizon. **Filtering problem** : $r = 0$ Estimate $x(N)$, given the observations of $y(t)$ from $t = 1$ to time $t = N$.

And in order to arrive at the algorithm, we have also to introduce two elements :

$$e(N) = y(N) - \hat{y}(N|N-1)$$

This is a very important term i.e **innovation**, that means the part of $y(N)$ that we cannot compute using $N-1$ data.

$$\nu(N) = x(N) - \hat{x}(N|N-1)$$

This is the **state prediction error**.

Now, with some mathematical and algebrical consideration we can write the algorithm :

1.2.1 Kalman filter algorithm

$$\left\{ \begin{array}{l} \hat{x}(N+1|N) = F\hat{x}(N|N-1) + \underbrace{FP(N)H^T(HP(N)H^T + V_2)^{-1}}_{\text{Predictor Gain}} e(N) \\ \hat{y}(N+1|N) = H\hat{x}(N+1|N) \\ e(N) = y(N) - \hat{y}(N|N-1) \\ \nu(N) = x(N) - \hat{x}(N|N-1) \\ P(N) = E[\nu(N)\nu(N)^T] \end{array} \right. \quad (1.2)$$

Looking closely at this formulation, two fundamental observations arise:

- **Comparison with Luenberger:** Just like a Luenberger observer, the new estimate is computed by adding a correction term to the model prediction. This correction is proportional to the innovation $e(N)$ multiplied by a specific Gain. However, while the Luenberger gain is constant and assigned by design, the Kalman Gain is **time-varying** and dynamically optimal.
- **The Covariance issue:** The algorithm in this form cannot be directly implemented yet. To compute the Gain, we need the state error prediction variance $P(N)$. While the initial condition $P(1)$ is known, future values of $P(N)$ depend on the unmeasurable true state $x(N)$. Therefore, a recursive mathematical method is required to compute $P(N)$ iteratively: this is achieved through the Discrete Algebraic Riccati Equation (DARE).

Once the forward filtering pass is fully computed up to the final sample, generating the causal estimates and the sequence of covariance matrices P , the

limits of real-time estimation have been reached. To further minimize the estimation error in an offline scenario, we must introduce the backward pass.

Capitolo 2

What is the kalman smoother

2.1 The Concept of Smoothing

As previously discussed, the Kalman filter optimally solves the estimation problem using a causal approach. To formally introduce the concept of smoothing, it is useful to define a generalized notation for state estimates.

Let $\hat{x}(k|j)$ denote the optimal estimate of the state x at time k , conditioned on all the measurements available up to and including time j . Using this notation, we can formally classify the different estimation paradigms based on the relationship between the time of estimation k and the time of the last available measurement j :

- **Prediction:** When $j < k$. For instance, $\hat{x}(k|k-1)$ is the *a priori* estimate (one-step-ahead prediction).
- **Filtering:** When $j = k$. Thus, $\hat{x}(k|k)$ is the *a posteriori* estimate, utilizing measurements up to the current moment.
- **Smoothing:** When $j > k$. In this scenario, the estimate of the state at time k incorporates measurements obtained *after* time k .

The core philosophy of smoothing is that "future" data contains valuable information about the past state. By using non-causal information, a smoothing algorithm can significantly reduce the estimation error covariance compared to a standard filter.

2.2 The Three Types of Smoothing Problems

Depending on how data is collected and how the time indices k and j are constrained, the literature categorizes smoothing into three distinct types:

1. **Fixed-Interval Smoothing:** The total number of available measurements N is fixed, and the goal is to compute the optimal estimate $\hat{x}(k|N)$ for all time steps $k \in [1, N]$. This is a strictly offline approach, typically used for post-mortem analysis of recorded data (such as the biomedical signals in this project).
2. **Fixed-Lag Smoothing:** The estimation delays the measurement by a constant number of steps L (called the lag). The algorithm continuously estimates $\hat{x}(k|k + L)$ as new data rolls in.
3. **Fixed-Point Smoothing:** The estimation time index k is held constant, while the measurement index j keeps increasing. The algorithm updates $\hat{x}(k|j)$ for $j = k + 1, k + 2, \dots$, focusing on reconstructing the state at one specific critical moment.

Capitolo 3

Algorithm and Implementation

3.1 The Rauch–Tung–Striebel (RTS) Smoother: A Bayesian Perspective

Since the total length of our electroencephalographic (EEG) recordings is known and available prior to processing, our challenge perfectly matches the **Fixed-Interval Smoothing** problem. Among the various algorithms available, the **Rauch–Tung–Striebel (RTS)** smoother stands out as the most efficient implementation.

To understand the mathematical foundation of the RTS smoother, it is highly beneficial to look at it through the lens of Bayesian estimation. From a probabilistic standpoint (as detailed in Särkkä’s Bayesian Filtering framework), the smoothing problem consists of computing the marginal posterior distribution of the state x_k given the entire measurement sequence $y_{1:N}$:

$$p(x_k|y_{1:N}) \tag{3.1}$$

By exploiting the Markov property of the state-space model—which states that the future is independent of the past given the present state—the joint probability equations can be factorized. The smoothing distribution can be derived by conditioning on the state at the next time step, x_{k+1} , leading to the fundamental backward recursive integral:

$$p(x_k|y_{1:N}) = p(x_k|y_{1:k}) \int \left[\frac{p(x_{k+1}|x_k)p(x_{k+1}|y_{1:N})}{p(x_{k+1}|y_{1:k})} \right] dx_{k+1} \tag{3.2}$$

Notice that this equation conceptually links three main components:

- $p(x_k|y_{1:k})$: The filtering distribution obtained during the forward pass.
- $p(x_{k+1}|x_k)$: The dynamic model transition probability.
- $p(x_{k+1}|y_{1:N})$: The smoothed distribution evaluated at the *next* time step $k + 1$.

3.1.1 The RTS Algorithm Formulation

When the dynamic system is linear and the noise is Gaussian, all the probability density functions in the Bayesian integral are Gaussian. Consequently, computing the exact posterior distribution reduces to finding its mean vector $\hat{x}(k|N)$ and its covariance matrix $P(k|N)$.

The RTS smoother executes this through a computationally elegant two-pass architecture. First, a standard Kalman filter forward sweep is executed over the entire dataset to compute and store the sequences of causal estimates $\hat{x}(k|k-1)$, $\hat{x}(k|k)$ and their respective error covariance matrices $P(k|k-1)$, $P(k|k)$ for all $k \in [1, N]$.

Once the final sample N is reached, the boundary condition is set ($\hat{x}(N|N) = \hat{x}(N)$ and $P(N|N) = P(N)$). From here, the RTS algorithm performs a second pass, sweeping **backward in time** (from $k = N - 1$ down to 1) using the following recursive algebraic equations derived from the Gaussian integrals:

$$G(k) = P(k|k)F^T P(k+1|k)^{-1} \quad (3.3)$$

$$\hat{x}(k|N) = \hat{x}(k|k) + G(k) [\hat{x}(k+1|N) - \hat{x}(k+1|k)] \quad (3.4)$$

$$P(k|N) = P(k|k) + G(k) [P(k+1|N) - P(k+1|k)] G(k)^T \quad (3.5)$$

Where $G(k)$ is the *Smoothing Gain*.

In optimal estimation theory (following Simon's formulation), the matrix $G(k)$ acts similarly to the Kalman gain, but in reverse. It optimally weights the new information coming "from the future". The physical interpretation is deeply intuitive: the optimal smoothed estimate $\hat{x}(k|N)$ is computed by taking the

causal filtered estimate $\hat{x}(k|k)$ and adding a correction vector. This correction is directly proportional to the difference between the actual smoothed state at the next time step and the *a priori* prediction that the forward filter originally made for that same step.

Capitolo 4

Practical Project

The project under consideration involves the use of EEG signals, which stands for electroencephalographic signals, or signals originating from the brain. The study focuses specifically on a particular phase, namely the phase shift between ERD (Event-Related Desynchronization) and ERS (Event-Related Synchronization) signals, which can occur, for example, when a movement or sensory stimulus occurs. Studying this sudden shift is not so simple, as one might think that the application of the FFT (Fast Fourier Transform) would suffice, but this cannot be effective in the presence of non-stationary signals (which is, in fact, the characteristic of the signals being analyzed).

Therefore, the Kalman filter and the smoother are used, both of which, however, are not used in their native application, but to best estimate the coefficients of a time-varying autoregressive model describing the signal.

The Kalman filter provides a recursive estimate of these coefficients using observations up to the current time, while the Rauch–Tung–Striebel Smoother refines them by also exploiting future observations. The resulting coefficients are finally used to reconstruct a more stable and accurate time-varying spectrum, allowing for a more reliable analysis of transient phenomena such as ERS and ERD.

So it's important to understand that in this project, the Rauch–Tung–Striebel Smoother is applied to an EEG analysis problem. This choice is not the algorithm's only application, but it provides a significant example of how a smoothing technique can be used to improve the estimation of a hidden state, in this case represented by the coefficients of a time-varying autoregressive

model.

4.1 Overview

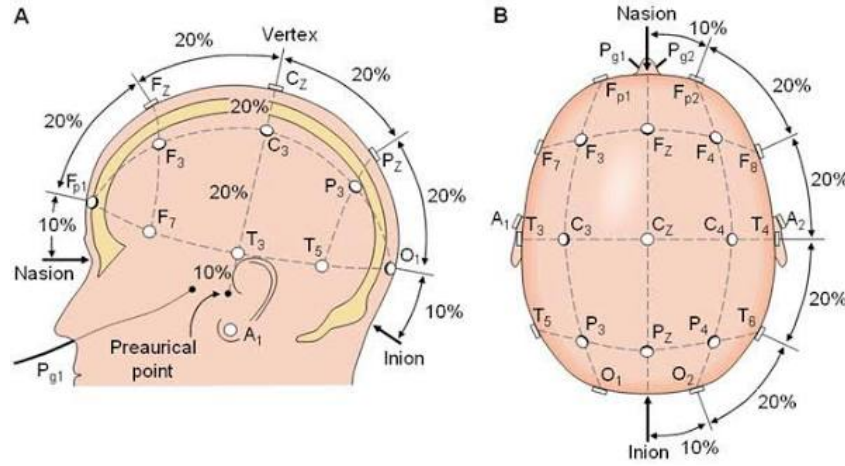


Figura 4.1: EEG scheme

In this image, it's possible to see how the electrodes are positioned on the head during the EEG. Between sensors 01 and 02 is the one used for this paper, namely Oz. This is located at the back of the skull, just above the occipital visual cortex, and therefore above the vision control center. Since the goal is to analyze behavior when the eyes close, we will use this data. As I mentioned, Oz is not explicitly present in the image because z stands for the midline, or central line, so it is on the line between O1 and O2.

4.1.1 Data download and cleaning

The first thing to do is find the data to be examined. The physionet website, in particular, has been very useful.

This provides real data, which is therefore already subject to noise. Before being used, it must be cleaned, including in terms of frequency, and then a band-pass filter is applied.

```
1 % Definiamo gli URL diretti ai file sul server di PhysioNet
2 url_run1 = 'https://archive.physionet.org/pn4/eegmidb/S001/
  S001R01.edf';
3 url_run2 = 'https://archive.physionet.org/pn4/eegmidb/S001/
  S001R02.edf';
```

```

4
5 file1 = 'S001R01.edf';
6 file2 = 'S001R02.edf';
7
8 options = weboptions('Timeout',60); % aspetta fino a 60
    secondi perch il sito lento e i classici 5 secondi non
    bastavano
9
10 % Controlla se i file esistono gi , altrimenti li scarica
11 if ~isfile(file1)
12     disp(['Scaricamento di ', file1, ' in corso... (potrebbe
    volerci qualche secondo)']);
13     websave(file1, url_run1);
14     disp('Download completato!');
15 end
16
17 if ~isfile(file2)
18     disp(['Scaricamento di ', file2, ' in corso...']);
19     websave(file2, url_run2);
20     disp('Download completato!');
21 end
22
23 % Run 1 = Occhi Aperti (Baseline 1)
24 % Run 2 = Occhi Chiusi (Baseline 2)
25
26 [data_open, ~] = edfread(file1);
27 [data_closed, ~] = edfread(file2);
28
29 % Estrazione del canale Oz che matlab ha rinominato Oz__ e
    conversione in un vettore numerico
30 y_open = vertcat(data_open.Oz__{:});
31 y_closed = vertcat(data_closed.Oz__{:});
32
33 y_raw = [y_open; y_closed];
34
35 % Otteniamo la frequenza di campionamento (PhysioNet usa 160 Hz
    )
36 fs = 160;
37 N = length(y_raw);
38 t_asse = (0:N-1) / fs; % Asse dei tempi in secondi

```

```

39
40 y_centered = y_raw - mean(y_raw);
41
42 %Filtraggio Bandpass (1 - 40 Hz)
43 % Eliminiamo le derivate lentissime (sudore/respiro) e il rumore
   ad alta frequenza
44 y_clean = bandpass(y_centered, [1 40], fs);

```

Listing 4.1: download and cleaning data

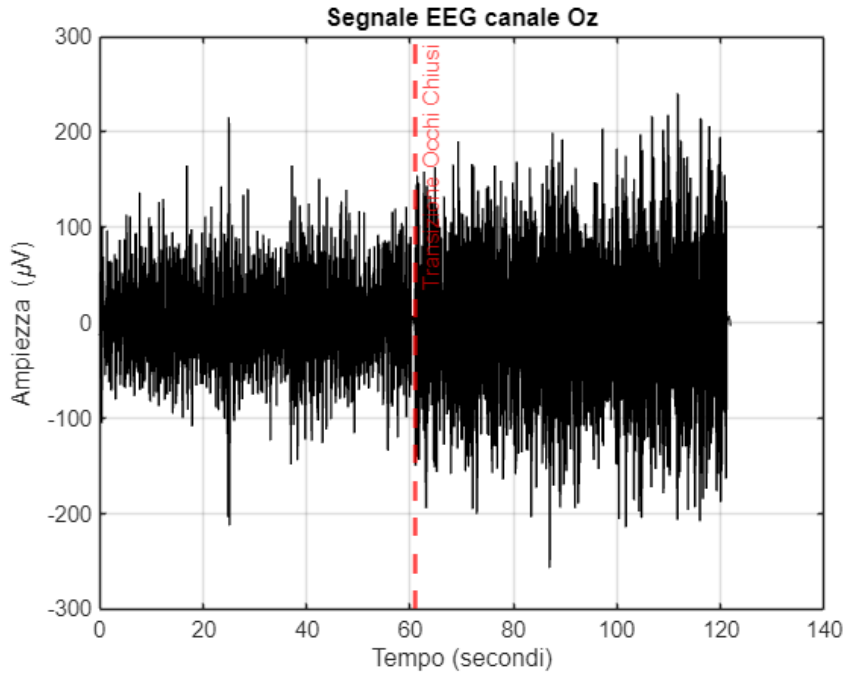


Figura 4.2: Signal

4.1.2 Implementation of kalman filter

To track the non-stationary dynamics of the EEG signal, we model the signal y_k as a time-varying Autoregressive process of order p , denoted as $AR(p)$. Mathematically, the signal at time k is described by a linear combination of its p past values plus a white noise term v_k :

$$y_k = a_1(k)y_{k-1} + a_2(k)y_{k-2} + \dots + a_p(k)y_{k-p} + v_k \quad (4.1)$$

where $a_1(k), a_2(k), \dots, a_p(k)$ are the time-varying AR coefficients. Based on empirical tuning, an order $p = 6$ was chosen to adequately capture the frequency components of the brain rhythms without incurring overfitting.

To estimate these dynamic coefficients using the Kalman Filter, we must translate the AR(p) model into a state-space representation. We define the state vector x_k as the collection of the AR coefficients at time k :

$$x_k = \begin{bmatrix} a_1(k) \\ a_2(k) \\ \vdots \\ a_p(k) \end{bmatrix} \quad (4.2)$$

Assuming that the parameters evolve according to a random walk model, the complete state-space formulation becomes:

$$x_k = x_{k-1} + w_k \quad (4.3)$$

$$y_k = H_k x_k + v_k \quad (4.4)$$

Here, the observation matrix $H_k = [y_{k-1}, y_{k-2}, \dots, y_{k-p}]$ contains the past p signal samples. The process noise w_k (with covariance matrix Q) drives the parameter variation, while v_k (with variance R) represents the measurement noise.

The covariance matrices were carefully tuned: we set $R = 0.1$ due to the inherent noisiness of the EEG recordings, and $Q = 10^{-6}I$. This specific value of Q acts as a "Goldilocks" parameter: it is large enough to allow the filter to adapt to the brain's state changes (such as the ERD/ERS transition), but small enough to prevent the spectral estimation from becoming erratic.

```

1 y = y_clean; % Questo      il vettore osservazioni reale
2
3 %p = 2
4 %provo ad aumentare l'ordine del modello perch  con p = 2 non
   si ha una
5 %buona visualizzazione di quello che accade
6 p = 6;
7 x = zeros(p, 1);          % Stima parametri AR
8 P = eye(p);               % Covarianza dell'errore di stima
9 %Q = 0.0001 * eye(p);     % Rumore di processo
10 %riduco Q perch          come se mi fidassi maggiormente del
    processo, questa
11 %considerazione          stata fatta dopo aver osservato maggiore
    rumore

```

```

12 %proveniente dalle misurazioni
13 % Q = 1e-4 * eye(p);
14 Q = 1e-6 * eye(p);
15 %R = 0.01; % Rumore di misura
16 %mentre aumento P perch mi fido meno delle misurazioni che
    come si visto
17 %sono molto rumorose
18 R = 0.1;
19 A = eye(p); % A = I (i parametri AR sono
    costanti) o molto molto lenti
20
21 % ci serve salvare le info per poi darle in pasto allo smoother
22 m_saved = zeros(p, N);
23 P_saved = zeros(p, p, N);
24
25 m_pred_saved = zeros(p, N);
26 P_pred_saved = zeros(p, p, N);

```

Listing 4.2: parameters setting

```

1 K_saved = zeros(p, N); % Matrice per salvare il guadagno di
    Kalman (6 x N)
2 e_saved = zeros(1, N); % Vettore per salvare l'innovazione (1 x
    N)
3
4 for k = (p+1):N
5     %H dipende dai dati reali in quell'istante
6     %H = [y(k-1), y(k-2)];
7
8     for i = 1:p
9         H(i) = y(k-i);
10    end
11
12    %Time update
13    x_pred = A * x;
14    P_pred = A * P * A' + Q;
15
16    innovazione = y(k) - (H * x_pred);
17    S = H * P_pred * H' + R;
18    K = P_pred * H' / S; % Guadagno di Kalman
19

```

```

20     x = x_pred + K * innovazione; % Aggiornamento stato
21     P = (eye(p) - K * H) * P_pred; % Aggiornamento covarianza
22
23     K_saved(:, k) = K;
24     e_saved(k) = innovazione;
25
26     %salvo i dati per lo smoother
27     m_saved(:, k) = x;
28     P_saved(:, :, k) = P;
29
30     m_pred_saved(:, k) = x_pred;
31     P_pred_saved(:, :, k) = P_pred;
32
33 end

```

Listing 4.3: kalman filter

4.1.3 Implementation of the smoother

Basically for the implementation of the smoother we need to follow the algorithm :

```

1 m_smoothed = zeros(p, N);
2 P_smoothed = zeros(p, p, N);
3
4 % L'ultimo punto uguale a quello filtrato
5 m_smoothed(:, N) = m_saved(:, N);
6 P_smoothed(:, :, N) = P_saved(:, :, N);
7
8 % Backward pass (da N-1 fino a p+1)
9 for k = (N-1):-1:(p+1)
10     %calcolo del guadagno di smoothing (G)
11     %G = P_k * A' * inv(P_pred_k+1)
12     G = P_saved(:, :, k) * A' / P_pred_saved(:, :, k+1);
13
14     %Aggiornamento media smussata
15     m_smoothed(:, k) = m_saved(:, k) + G * (m_smoothed(:, k+1)
16     - m_pred_saved(:, k+1));
17
18     %Aggiornamento covarianza smussata

```

```

18     P_smoothed(:, :, k) = P_saved(:, :, k) + G * (P_smoothed(:,
19     :, k+1) - P_pred_saved(:, :, k+1)) * G';
    end

```

Listing 4.4: kalman smoother

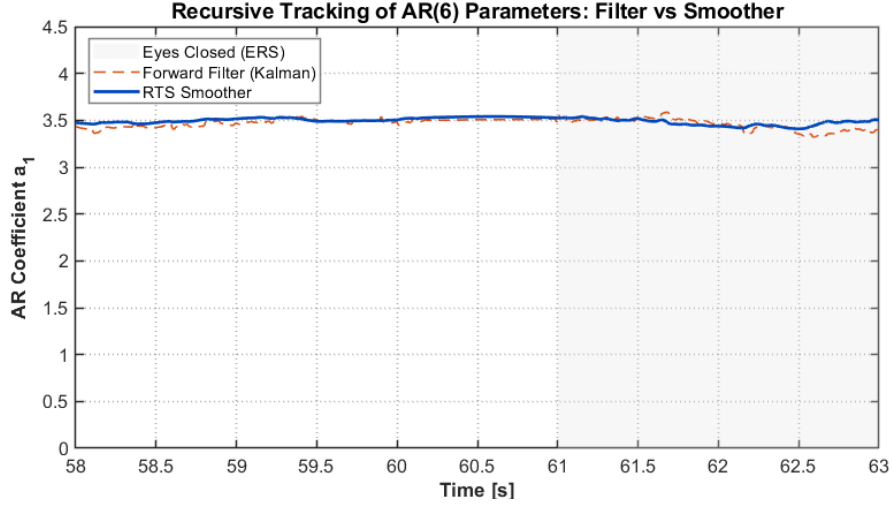


Figure 4.3: Filter vs smoother

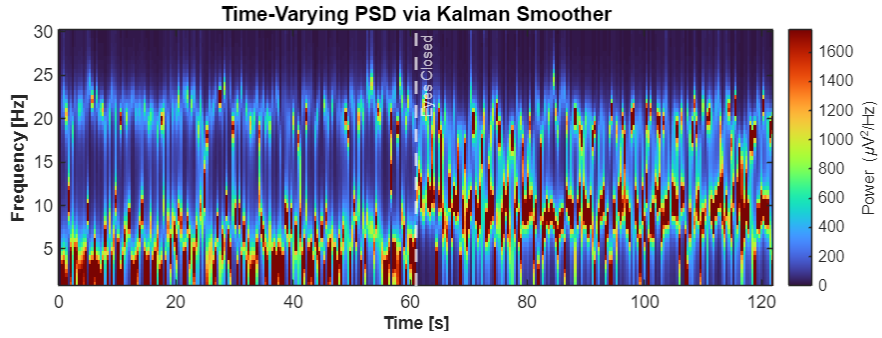


Figure 4.4: Power spectral density

4.1.4 Graph discussion

The effectiveness of the proposed architecture is evaluated through visual analyses that highlight both the algorithmic stability of the tracking and the neurophysiological coherence of the results.

Recursive Tracking: Filter vs Smoother

Figure 4.3 perfectly visualizes the main theoretical limitation of a causal filter and the decisive contribution of the RTS smoother. Looking at the Kalman

Forward estimate (dashed orange line), we observe that the algorithm reacts somewhat abruptly to the noise present in the data (e.g., around $t = 61.27$ s, where a rapid drop occurs). Because the forward filter can only "see" the present and the past, it misinterprets this noise spike as a structural change in the system. Conversely, the RTS Smoother (solid blue line) processes the data backward and already knows the future trajectory of the signal. It correctly identifies the drop as a temporary artifact, completely ignoring the false peak and maintaining a stable, elegant, and highly accurate parameter estimate.

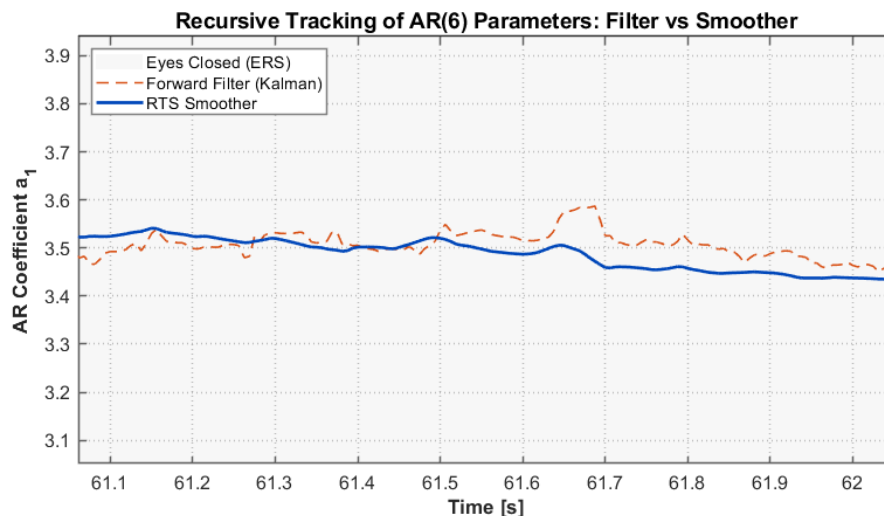


Figure 4.5: Filter vs smoother zoom

Time-Varying PSD (Spectrogram)

Figure 4.4 shows the final spectrogram, computed by evaluating the frequency response of the time-varying AR coefficients estimated by the smoother. This graph vividly illustrates the neurophysiological transition discussed in the introduction.

During the first 60 seconds (Eyes Open), the visual cortex is actively processing stimuli. The neurons fire asynchronously, producing a chaotic, low-amplitude signal. This is the **Event-Related Desynchronization (ERD)**, which is reflected in the spectrogram as a relatively flat spectrum without dominant high-power bands.

At exactly $t = 60$ s, the subject closes their eyes. Deprived of visual input, the neurons of the visual cortex begin to oscillate in unison. This macroscopic synchronization, known as **Event-Related Synchronization (ERS)**,

immediately triggers the **Alpha Rhythm**. The smoother successfully tracks this transition: right after the 60-second mark, a prominent, continuous, high-power red band emerges exactly between 8 and 12 Hz, perfectly confirming the expected physiological behavior.

To demonstrate that the filter and smoother are effective, I also calculated the PSD on the initial signal without first running the two Kalman instruments. To do this, however, I immediately encountered the first limitation: I had to know exactly that the change occurred at 60 s and therefore 10 Hz, otherwise I would not have been able to calculate the entire PSD, so I first calculated the one with eyes open and then the one with eyes closed. You can see the peak that confirms the structure. However, the limitation is precisely the one described initially: this graph is an average and therefore does not allow us to visualize a true evolution.

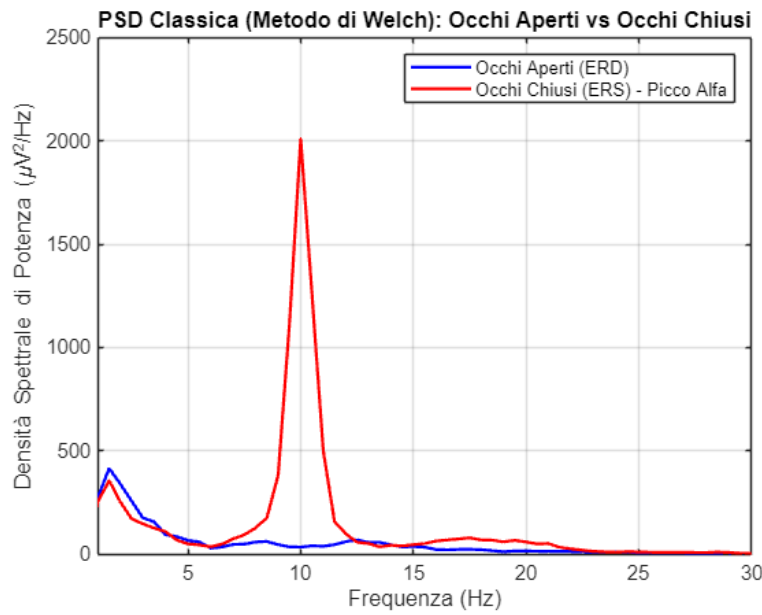


Figura 4.6: Psd on signal without kalman

Capitolo 5

Conclusion

In conclusion, it must be observed that RTS is an offline algorithm, therefore requiring the entire measurement sequence.

The performance of this algorithm depends directly on the Q and R matrices, which, as mentioned, specify the process and measurement noise. These matrices were adjusted empirically in the design, therefore conducting a series of tests until reaching an acceptable result. However, it should be noted that in many applications, this can be a significant limitation, so calibration could be considered using other methods. This also applies to the choice of the AR model order.

In the future, an algorithm of this type could also be applied to **multiple channels**, not just the Oz one, and even to non-linear systems with the implementation of an **extended Kalman filter**.

Bibliografia

- [1] Särkkä, Simo, and Lennart Svensson. Bayesian filtering and smoothing. Vol. 17. Cambridge university press, 2023.
- [2] Simon, Dan. Optimal state estimation: Kalman, H infinity, and nonlinear approaches. John Wiley Sons, 2006.
- [3] Bittanti, Sergio. Model identification and data analysis. John Wiley Sons, 2019.